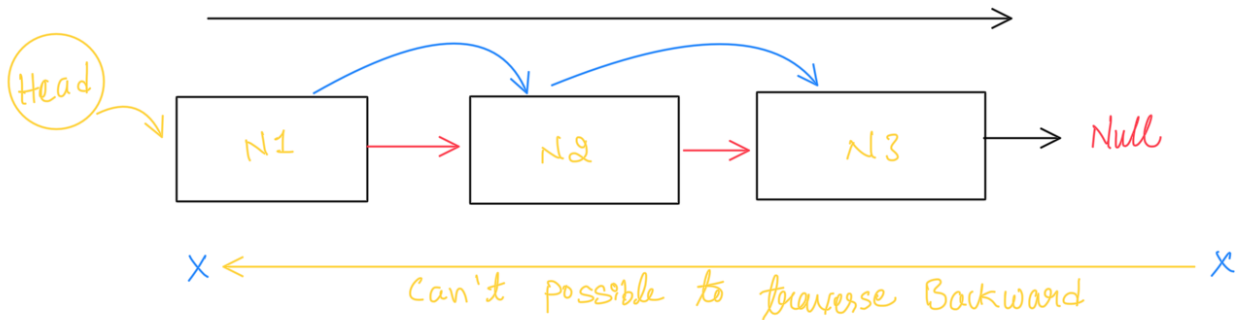


Doubly Linked List

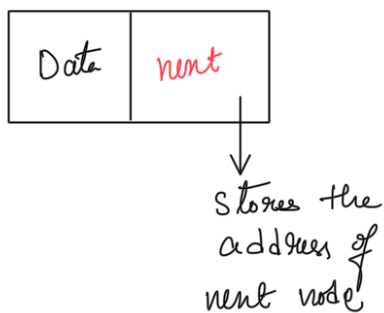
DAY 40
19/08/2025

As we observed that the Singly Linked List is unidirectional
We can traverse on Singly Linked List in single direction

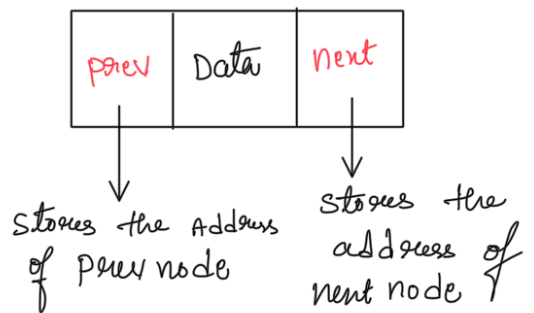


So Overcome this issue Doubly Linked List Data structure used

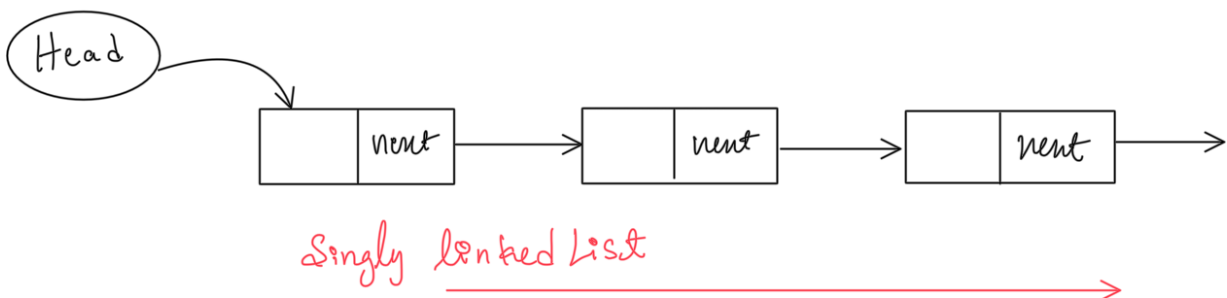
Node of Singly Linked List



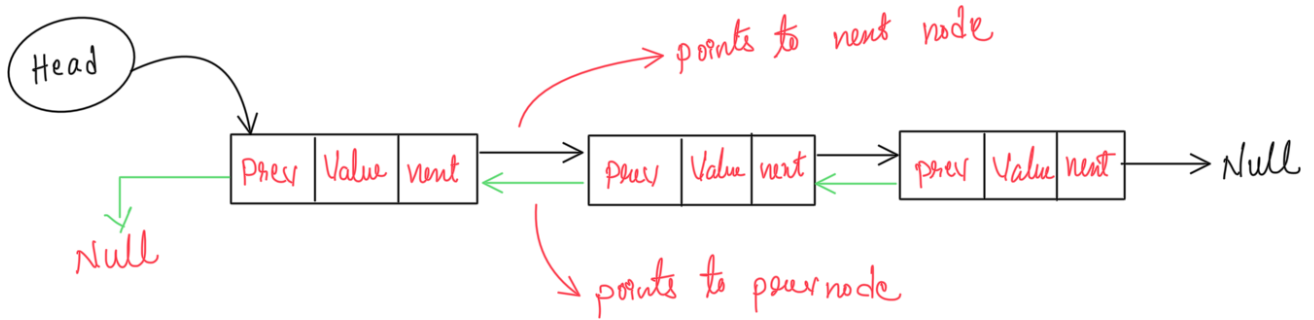
Node of Doubly Linked List



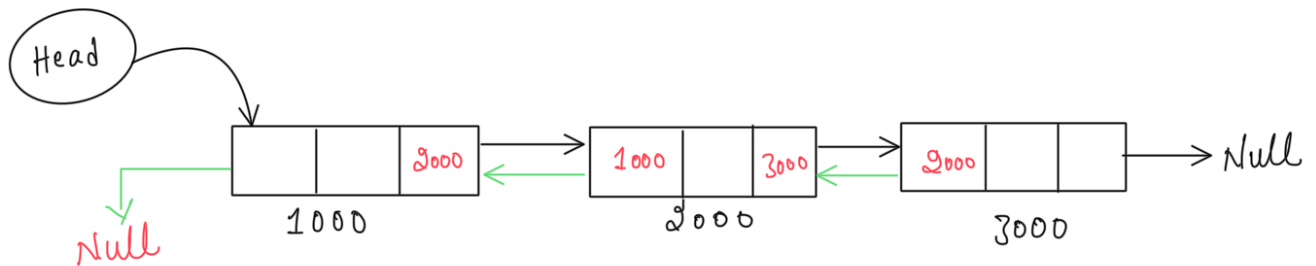
Singly Linked List



Doubly Linked List



conclusion $\Rightarrow$ Every Node Have address of *next node* and *prev Node*



prev address of HeadNode is pointing to Null
Because the Headnode doesn't contain any previous node, Head node is the first node

Last node next address is Null Bcz the Lastnode doesn't point to any next node, It's a last node of list.

To store prev address Extra memory is consumed
And have to deal with prev & next pointers.

managing Two pointers (prev, next)
prev address storing consumes Extra Space

Operations Performed on Doubly Linked List

Insert, Delete, Search, print

Insert

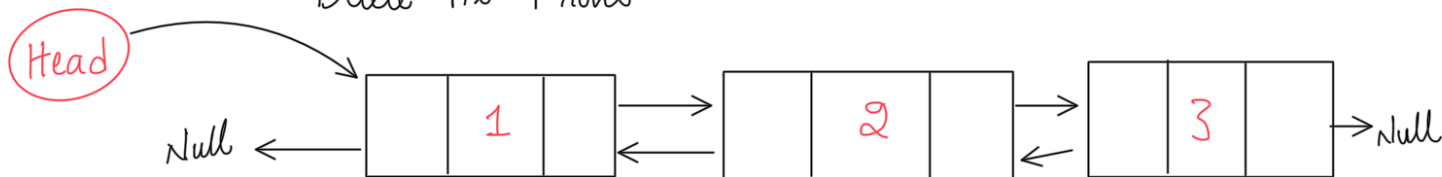
- Insert At Front
- Insert At End
- Insert At pos

Delete

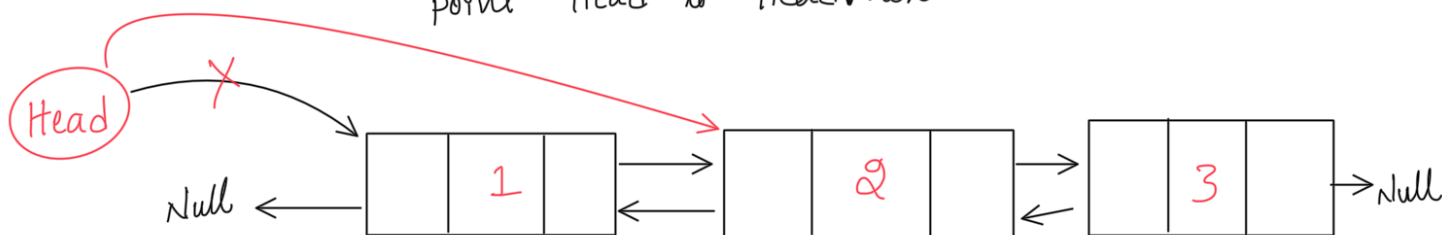
- Delete At Front
- Delete At End
- Delete At pos

Delete At Beginning

Delete At Front

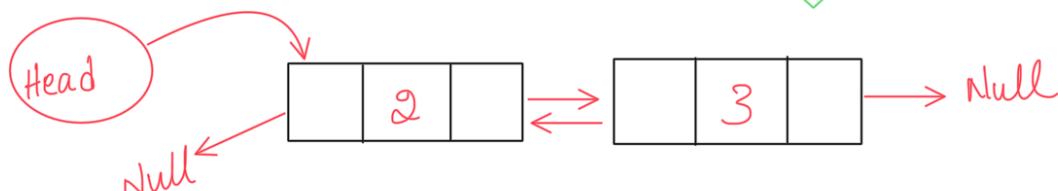
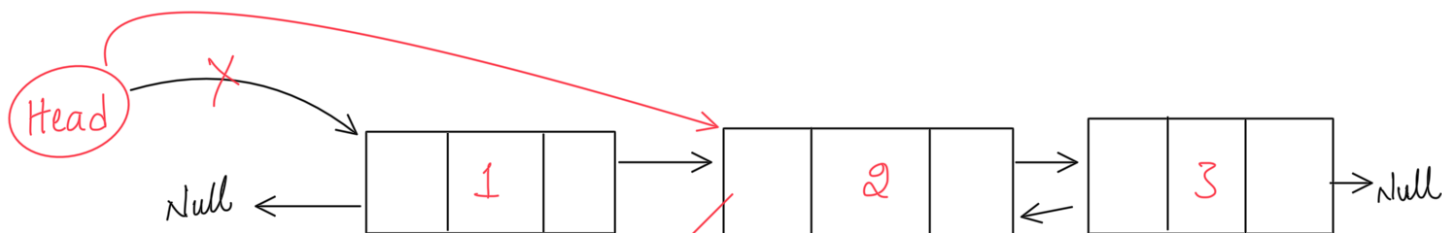


point Head to Head.next



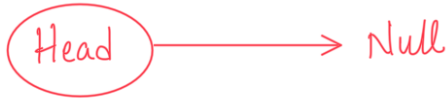
Head is updated
Now update prev of Head

Head.prev = null

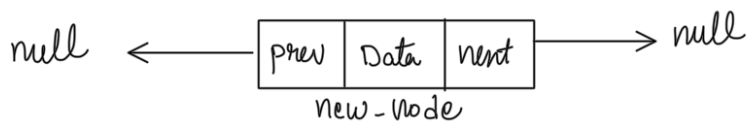


Insert At start

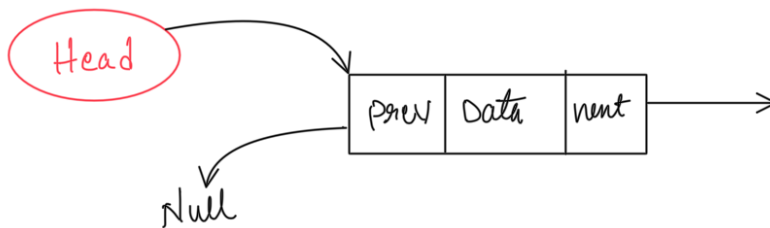
Initially Head is null Bcz list is empty



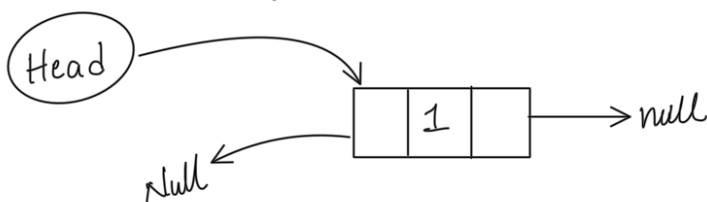
For Insertion we create newnode, the newnode next is prev to null



if Head = null then Just point Head to newnode
Head = new-node



if list is not Empty



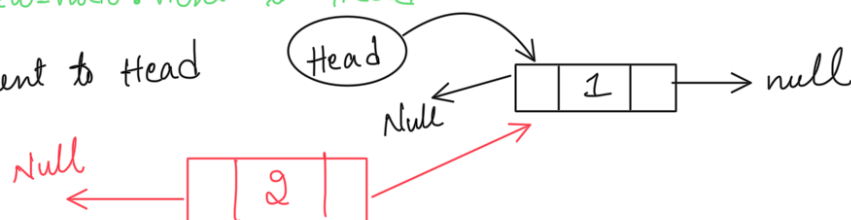
Insert new-node at front



step
①

point new-node.next to Head

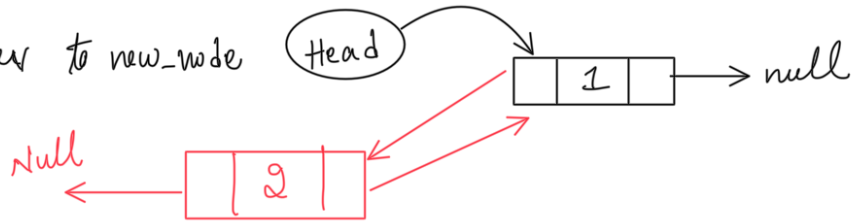
point new-node.next to Head



Step

②

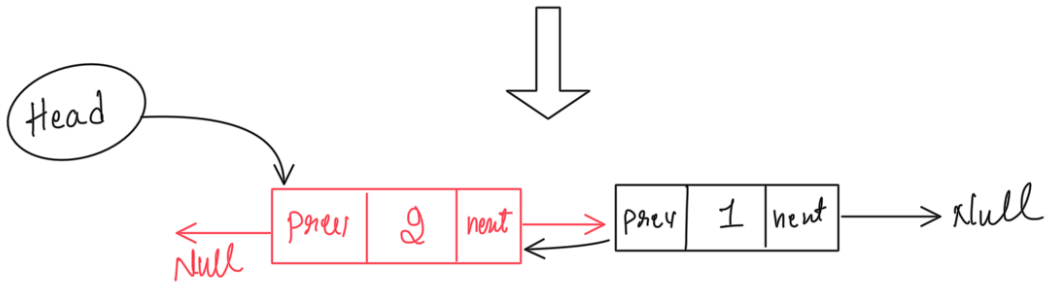
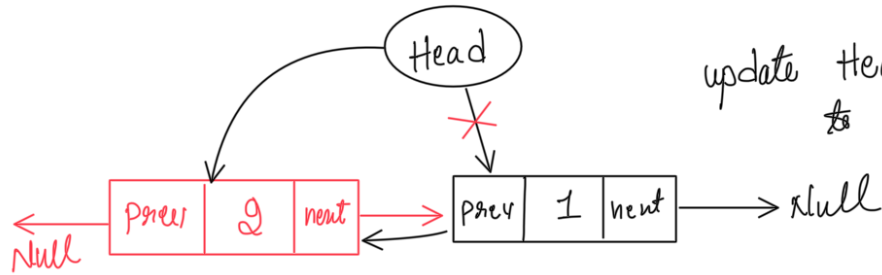
point Head.prev to new-node



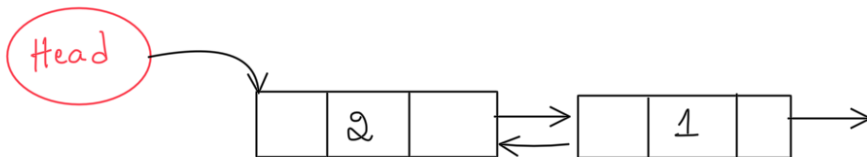
Step

③

update Head : point it to new-node



Insert At End



Insert At End



Step ① Search for Last node

Step ② Last node next newnode

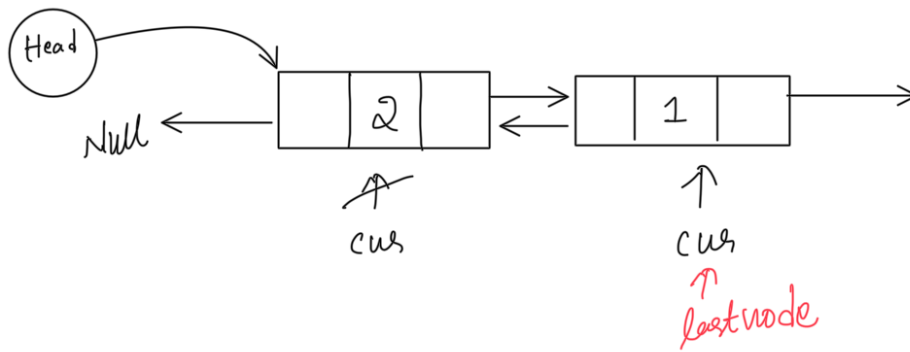
Step ③ New node prev Lastnode

Base Case (List Empty)

then

Head = new-node

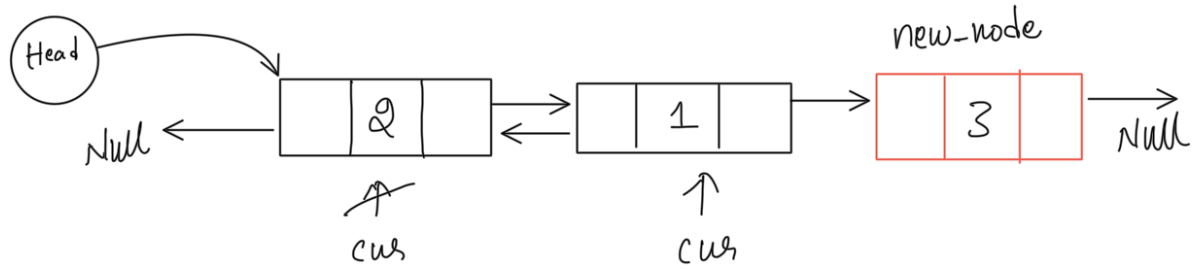
Step 1



point cur to last node

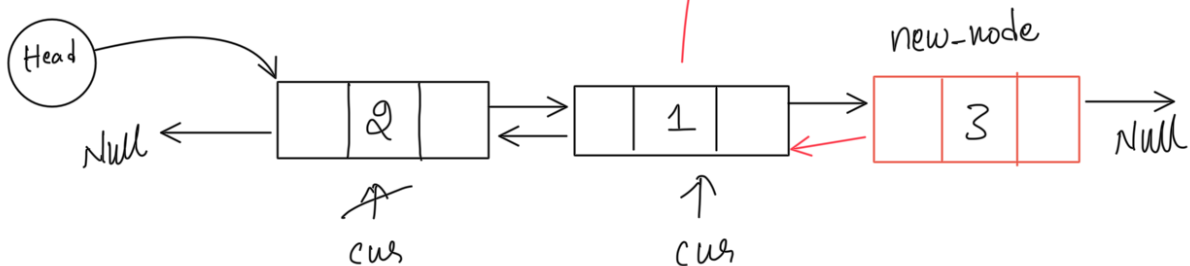
Step 2 last node . next = new-node

Step 1



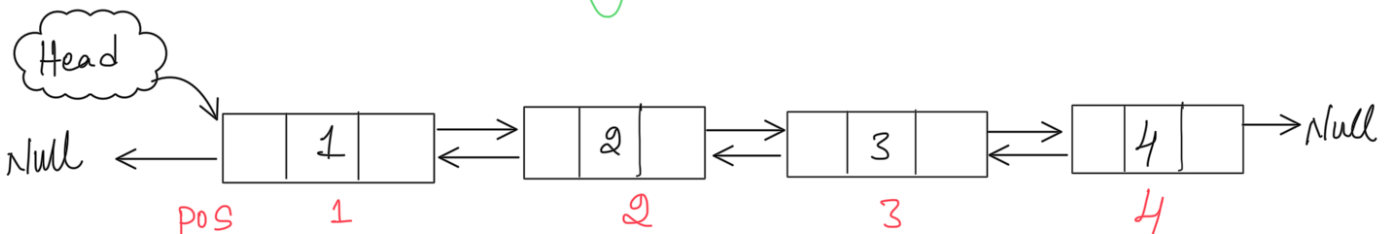
Step 3

new-node - prev = last node

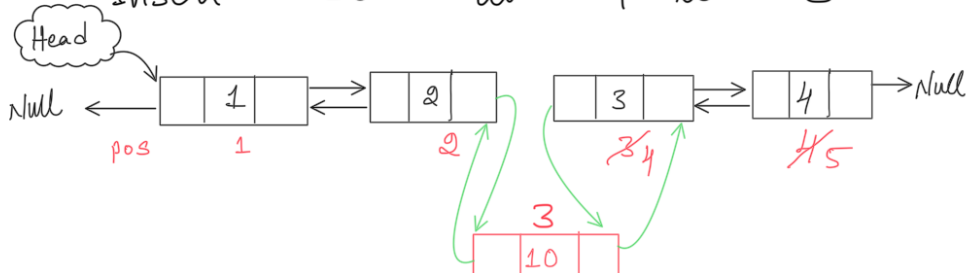


Insert At position

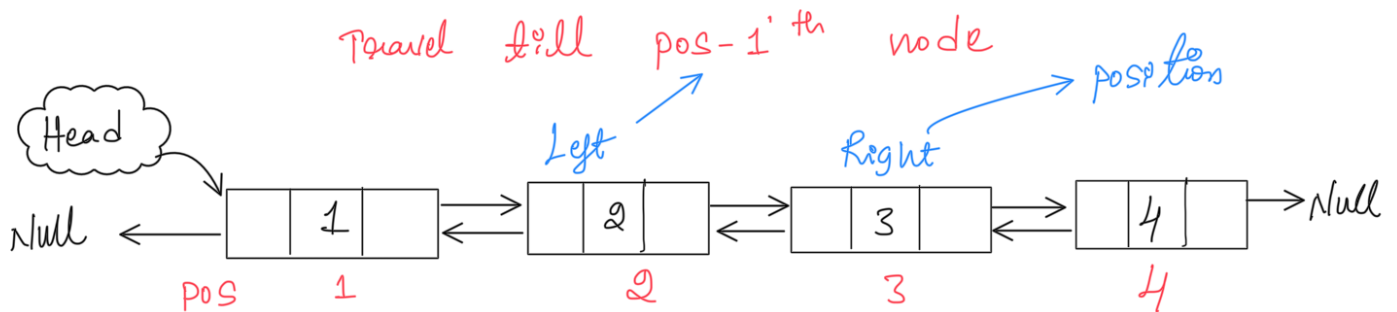
using 2 pointers left right



Insert 10 at position 3



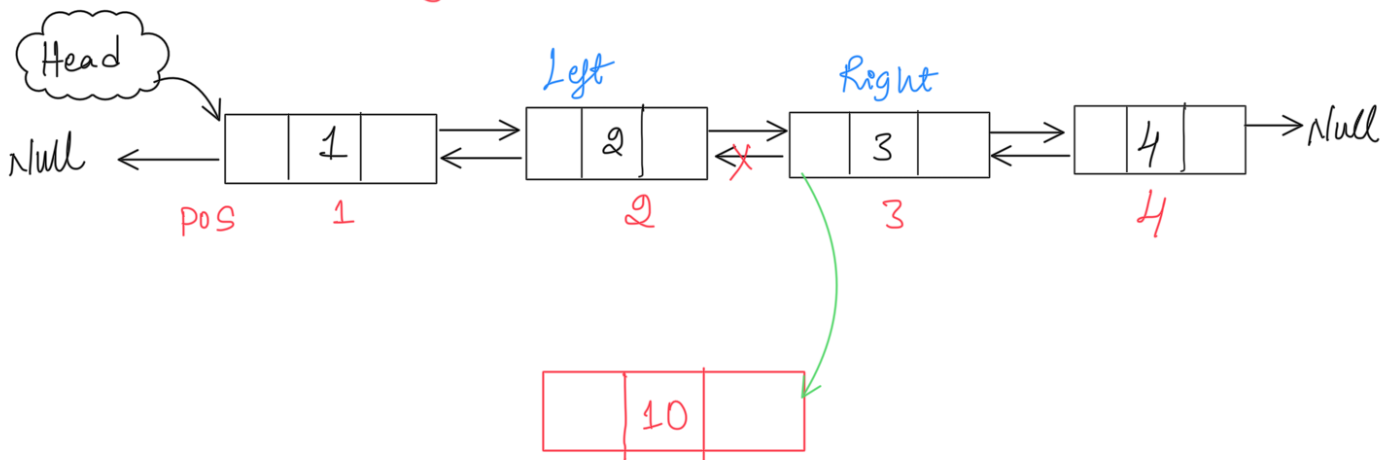
step 1 travel till prev node of insert position
OR



Insert 10 at position 3

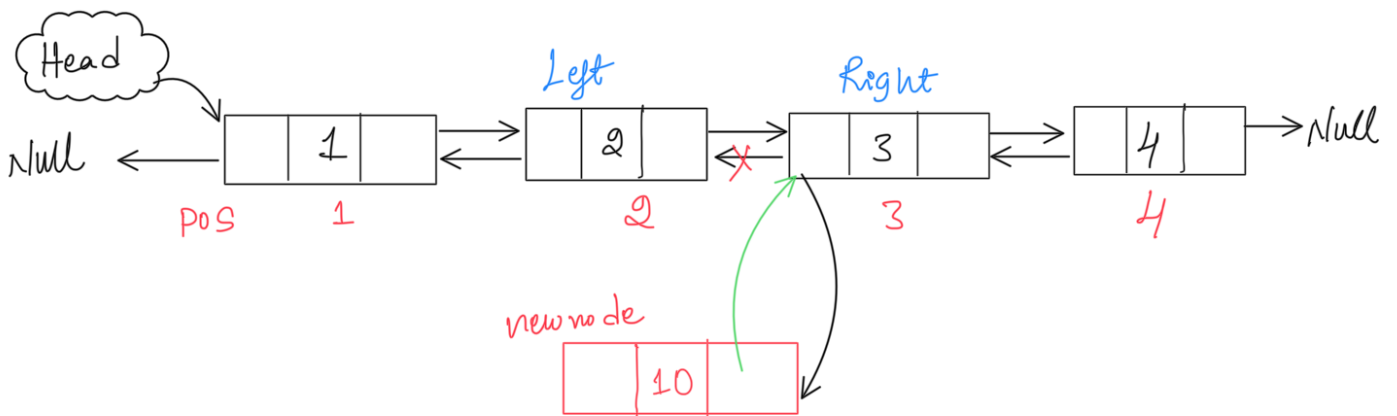
step 2

Right node. prev = new node



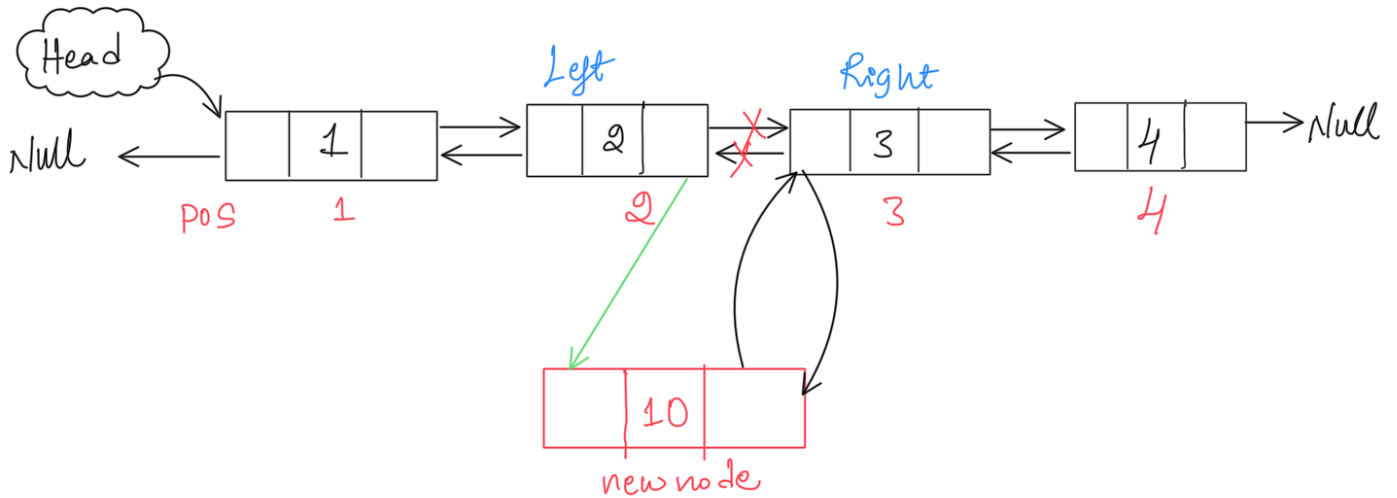
step 3

new Node. next = right node



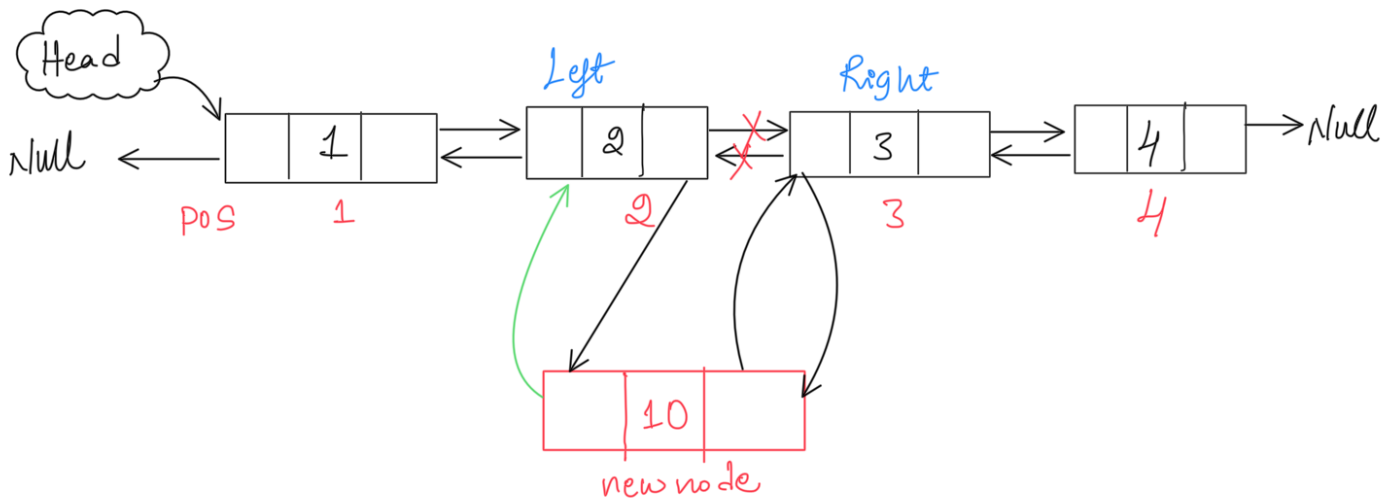
step 4

leftNode.next = newNode



step 5

newNode.prev = leftNode



Insert At position using Single pointers

prev.next = newNode

newNode.prev = prev

newNode.next = prev.next

prev.next.prev = newNode